

Hardware/Software Co-Design for MAYO Verification

Project Presentation

Kowsyap Pranay Kumar Musumuri

G01503265

ECE 615

Motivation

Why Post-Quantum Signatures?

The challenge? Quantum computers.

- They're powerful. They solve problems like **integer factorization** and **discrete logarithms** easily.
- The problem? These are the foundations of today's encryption.

The solution? Post-Quantum Cryptography (PQC).

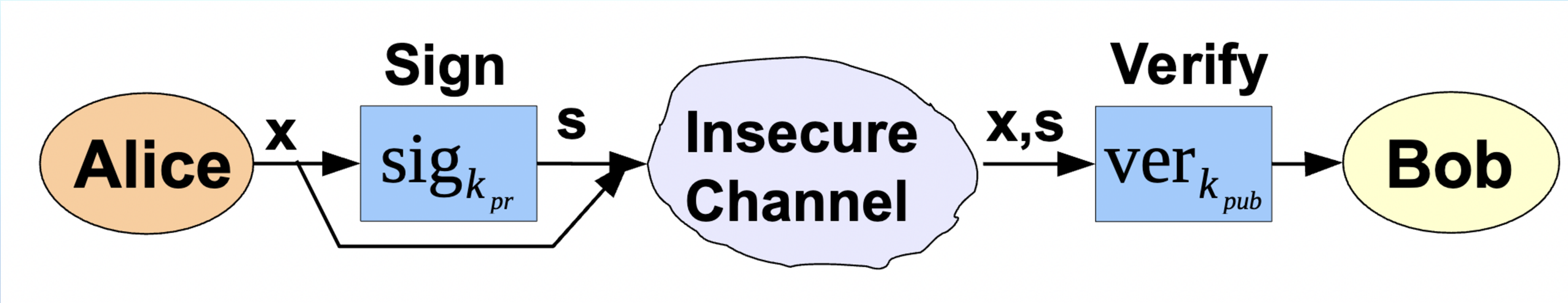
- PQC designs cryptographic systems that stay secure even if quantum computers become common.

MAYO Signature Scheme

What Is MAYO?

Parameter set security level	MAYO ₁ 1	MAYO ₂ 1	MAYO ₃ 3	MAYO ₅ 5
n	86	81	118	154
m	78	64	108	142
o	8	17	10	12
k	10	4	11	12
q	16	16	16	16
salt.bytes	24	24	32	40
digest.bytes	32	32	48	64
pk_seed.bytes	16	16	16	16
$f(z)$	$f_{78}(z)$	$f_{64}(z)$	$f_{108}(z)$	$f_{142}(z)$
secret key size	24 B	24 B	32 B	40 B
public key size	1420 B	4912 B	2986 B	5554 B
signature size	454 B	186 B	681 B	964 B
expanded sk size	141 KB	99 KB	367 KB	822 KB
expanded pk size	142 KB	104 KB	370 KB	828 KB

- MAYO is a multivariate post-quantum digital signature scheme
- Based on operations over small finite fields, especially GF(16)
- Designed to offer strong security with compact computations
- Signature verification involves structured matrix/vector operations and hashing



MAYO Parameters

Security Level 1 (Set 2) — R1 and R2 Instances

Parameter	Round 1 (R1)	Round 2 (R2)
m (equations / GF(16) outputs)	64	64
n (variables)	78	81
o (oil variables)	18	17
k (signature vectors)	4	4
cpk (bytes)	5,488 B	4,912 B
sig (bytes)	180 B	186 B
epk (bytes)	98,592 B	106,272 B
salt	24 B	24 B
digest	32 B	32 B
pk_seed	16 B	16 B

Verification Algorithm

Algorithm 6 MAYO.ExpandPK(cpk)

Input: Compact public key $\text{cpk} \in \mathcal{B}^{\text{cpk.bytes}}$

Output: Expanded public key $\text{epk} \in \mathcal{B}^{\text{epk.bytes}}$

```
1: //Parse cpk.
2:  $\text{seed}_{\text{pk}} \leftarrow \text{cpk}[0 : \text{pk\_seed.bytes}]$ 
3:
4: //Expand  $\text{seed}_{\text{pk}}$  and return.
5:  $\text{epk} = \text{AES-128-CTR}(\text{seed}_{\text{pk}}, \text{P1.bytes} + \text{P2.bytes}) \parallel \text{cpk}[\text{pk\_seed.bytes} : \text{pk\_seed.bytes} + \text{P3.bytes}]$ 
6: return epk.
```

Inputs: message, signature, public key

Output: valid / invalid

Main steps:

- decode **signature**-related values
- derive **target** ($\mathbf{t}$) value using SHAKE256
- expand **public-key**-related data using AES128 and then decode the data into m matrices of $n \times n$
- compute verification-side **GF(16) matrix/vector operations** of $n \times n$ matrices, $n = 81$
- **compare** computed result with target

Algorithm 8 MAYO.Verify(epk, M, sig)

Input: Expanded public key $\text{epk} \in \mathcal{B}^{\text{epk.bytes}}$

Input: Message $M \in \mathcal{B}^*$

Input: Signature $\text{sig} \in \mathcal{B}^{\text{sig.bytes}}$

Constant: $\mathbf{E} \in \mathbb{F}_q^{m \times m}$ // Represents multiplication by z in $\mathbb{F}_q[z]/(f(z))$

Output: An integer result to indicate if sig is valid (result = 0) or invalid (result < 0).

```
1: //Decode epk.
2:  $\text{P1\_bytestring} \leftarrow \text{epk}[0 : \text{P1.bytes}]$ 
3:  $\text{P2\_bytestring} \leftarrow \text{epk}[\text{P1.bytes} : \text{P1.bytes} + \text{P2.bytes}]$ 
4:  $\text{P3\_bytestring} \leftarrow \text{epk}[\text{P1.bytes} + \text{P2.bytes} : \text{P1.bytes} + \text{P2.bytes} + \text{P3.bytes}]$ 
5:  $\{\mathbf{P}_i^{(1)}\}_{i \in [m]} \leftarrow \text{Decode}_{\mathbf{P}^{(1)}}(\text{P1\_bytestring})$  //  $\mathbf{P}_i^{(1)} \in \mathbb{F}_q^{(n-o) \times (n-o)}$  upper triangular
6:  $\{\mathbf{P}_i^{(2)}\}_{i \in [m]} \leftarrow \text{Decode}_{\mathbf{P}^{(2)}}(\text{P2\_bytestring})$  //  $\mathbf{P}_i^{(2)} \in \mathbb{F}_q^{(n-o) \times o}$ 
7:  $\{\mathbf{P}_i^{(3)}\}_{i \in [m]} \leftarrow \text{Decode}_{\mathbf{P}^{(3)}}(\text{P3\_bytestring})$  //  $\mathbf{P}_i^{(3)} \in \mathbb{F}_q^{o \times o}$  upper triangular
8:
9: //Decode sig.
10:  $\text{salt} \leftarrow \text{sig}[\lceil nk/2 \rceil : \lceil nk/2 \rceil + \text{salt.bytes}]$ 
11:  $\mathbf{s} \leftarrow \text{Decode}_{\text{vec}}(kn, \text{sig})$ 
12: for  $i$  from 0 to  $k - 1$  do
13:    $\mathbf{s}_i \leftarrow \mathbf{s}[i * n : (i + 1) * n]$ 
14:
15: //Hash message and derive  $\mathbf{t}$ .
16:  $\text{M\_digest} \leftarrow \text{SHAKE256}(M, \text{digest.bytes})$  //  $\text{M\_digest} \in \mathcal{B}^{\text{digest.bytes}}$ 
17:  $\mathbf{t} \leftarrow \text{Decode}_{\text{vec}}(m, \text{SHAKE256}(\text{M\_digest} \parallel \text{salt}, \lceil m \log(q)/8 \rceil))$  //  $\mathbf{t} \in \mathbb{F}_q^m$ 
18:
19: //Compute  $\mathcal{P}^*(\mathbf{s})$ .
20:  $\mathbf{y} \leftarrow \mathbf{0}_m$  //  $\mathbf{y} \in \mathbb{F}_q^m$ 
21:  $\ell \leftarrow 0$ 
22: for  $i$  from 0 to  $k - 1$  do
23:   for  $j$  from  $k - 1$  to  $i$  do
24:      $\mathbf{u} \leftarrow \begin{cases} \left\{ \mathbf{s}_i^T \begin{pmatrix} \mathbf{P}_a^{(1)} & \mathbf{P}_a^{(2)} \\ \mathbf{0} & \mathbf{P}_a^{(3)} \end{pmatrix} \mathbf{s}_i \right\}_{a \in [m]} & \text{if } i = j \\ \left\{ \mathbf{s}_i^T \begin{pmatrix} \mathbf{P}_a^{(1)} & \mathbf{P}_a^{(2)} \\ \mathbf{0} & \mathbf{P}_a^{(3)} \end{pmatrix} \mathbf{s}_j + \mathbf{s}_j^T \begin{pmatrix} \mathbf{P}_a^{(1)} & \mathbf{P}_a^{(2)} \\ \mathbf{0} & \mathbf{P}_a^{(3)} \end{pmatrix} \mathbf{s}_i \right\}_{a \in [m]} & \text{if } i \neq j \end{cases}$  //  $\mathbf{u} \in \mathbb{F}_q^m$ 
25:      $\mathbf{y} \leftarrow \mathbf{y} + \mathbf{E}^\ell \mathbf{u}$ 
26:      $\ell \leftarrow \ell + 1$ 
27:
28: //Accept signature if  $\mathbf{y} = \mathbf{t}$ .
29: if  $\mathbf{y} = \mathbf{t}$  then
30:   return 0
31: return -1
```

Algorithm in detail

sig_decoder → s vectors
epk_decoder → P1, P2, P3 matrices
quad_accumulator → $Y = s^T \cdot P \cdot s$ in GF(16)

$$P_a = \begin{bmatrix} P1_a & P2_a \\ 0 & P3_a \end{bmatrix} \in \text{GF}(16)^{n \times n}$$

$P1_a \in \text{GF}(16)^{(n-o) \times (n-o)}$ upper triangular (vinegar variables)

$P2_a \in \text{GF}(16)^{(n-o) \times o}$

$P3_a \in \text{GF}(16)^{o \times o}$ upper triangular (o = oil variables)

$y \in \text{GF}(16)^m$

$P_a \in \text{GF}(16)^{n \times n}$

$s_i \in \text{GF}(16)^{n \times 1}$

for i = 0 to k-1:

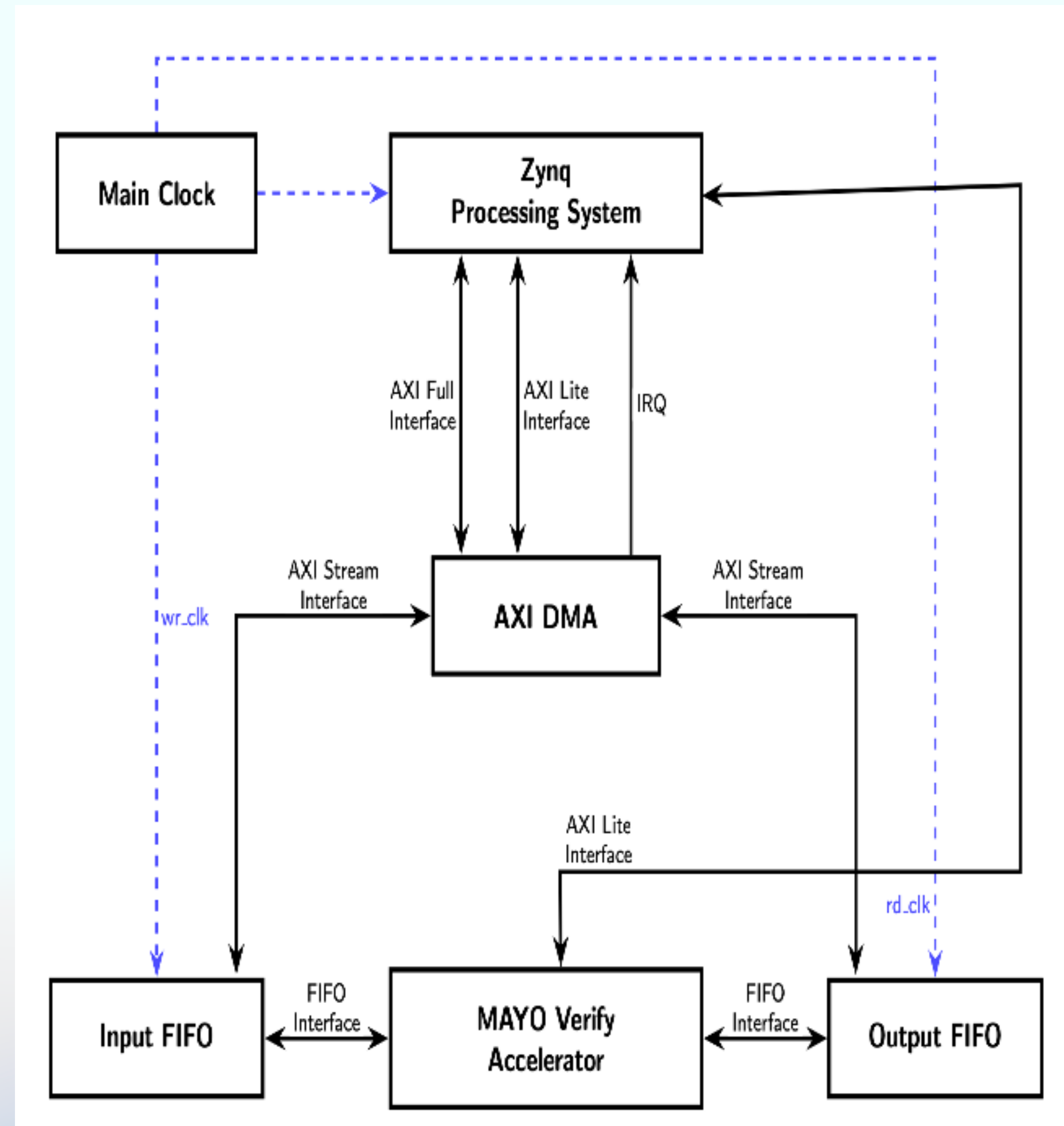
 for j = i to k-1:

$y \leftarrow y + (s_i^T \cdot P_a \cdot s_j)$ for all a ∈ [m]

$Y == T \rightarrow$ SIGNATURE VALID / INVALID

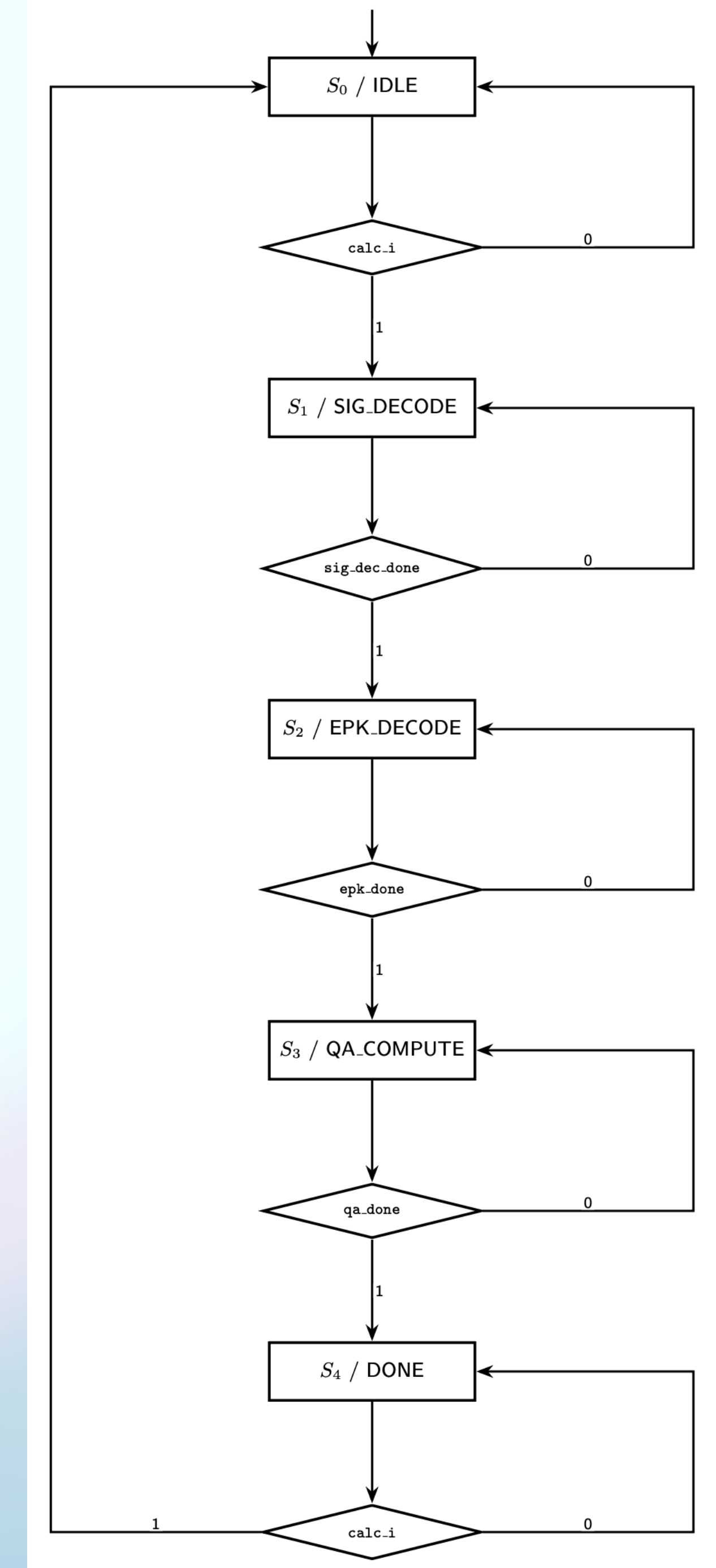
System Architecture

- Host PC → UART → Zynq PS (ARM Cortex-A9, bare-metal C)
- PS: mayo_expand_pk() — CPK → EPK via AES-128-CTR
- PS: deriveT() — SHAKE256(hash(msg) || salt) → target vector T
- PS → AXI DMA → streams SIG || EPK to PL
- PL: sig_decoder, epk_decoder, quad_acc → Y
- PS: compare Y == T → send 0x01 (PASS) / 0x00 (FAIL) via UART



Hardware Architecture

- AXI-Lite slave — bit 0: write 0x1 to start, read STATUS_DONE
- AXI Stream slave/master — receives SIG || EPK, emits Y result via DMA
- sig_decoder — unpacks $k=4$ compressed signature vectors (nibble alignment)
- epk_decoder — reads P_1, P_2, P_3 from stream into multi-bank SDP RAM
- quad_accumulator — computes $Y = s^T \cdot P \cdot s$ via parallel GF(16) multipliers



Hardware Details

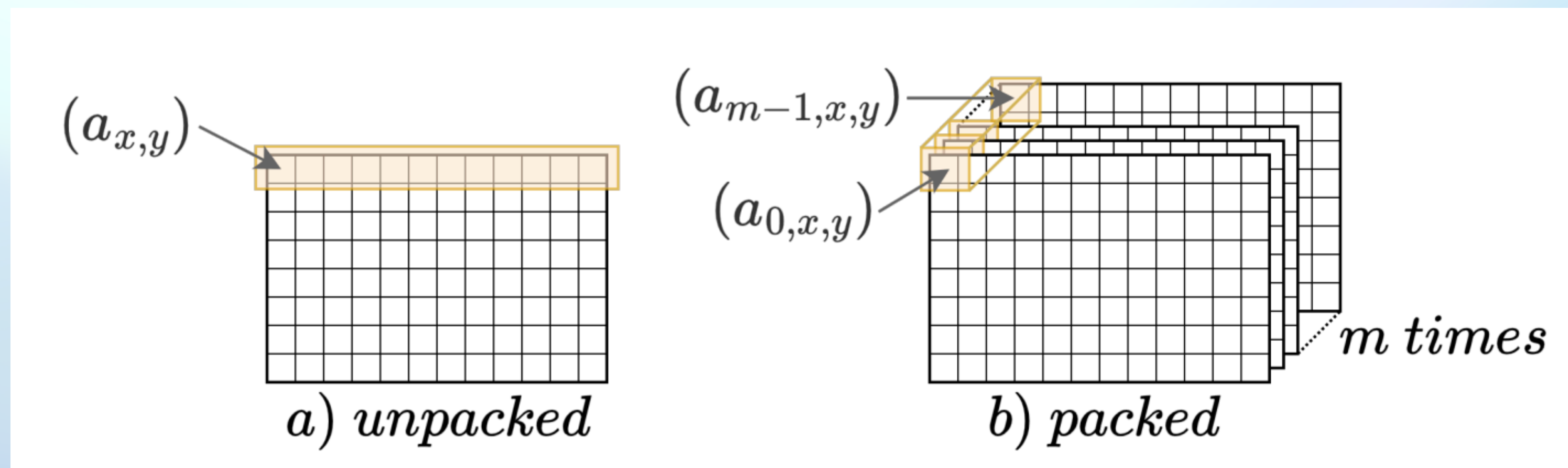
sig_decoder

- Receives 64-bit signature words via AXI Stream
- Accumulates nibbles into **s[k] register**
- Each register has $4 \times n \rightarrow 256$ bit width
- Uses SIPO FIFO for byte reassembly
- Asserts **sig_dec_done** when decoding is complete

Hardware Details

epk_decoder

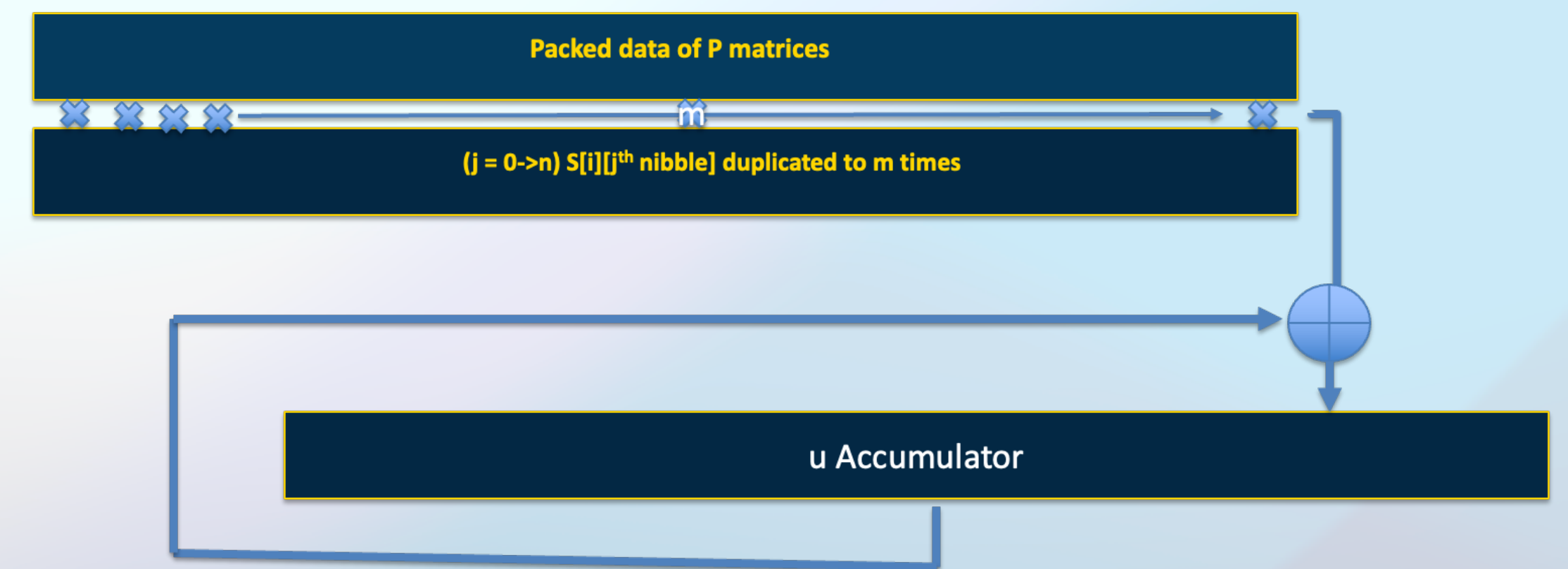
- Streams 64 bit expanded public key (**EPK**) words via AXI stream
- Constructs **P₁**, **P₂**, **P₃** matrices.
- P matrices are not stored directly, instead stored as **P.s** matrices but in packed format
- Each BRAM location stores **m elements** ($p_i(x, y), i = 1 \dots m$)



Hardware Details

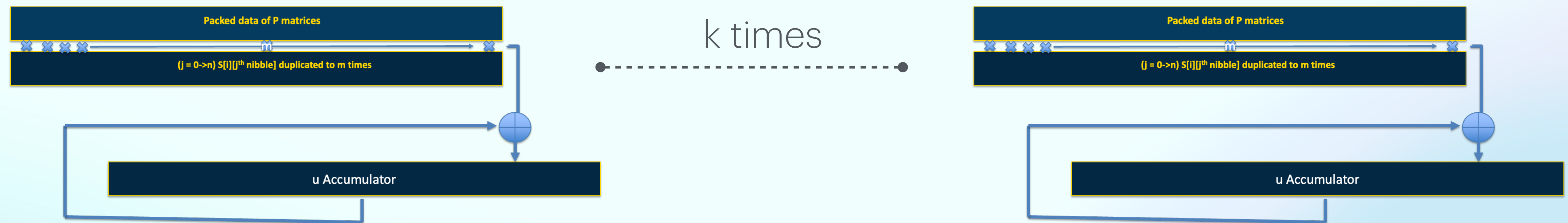
quad_accumulator

- Iterates over all **(i, j) s-pairs**
- **mul_gf16_array**: parallel GF(16) multiplication of s^T and $P.s$
- **reduce_mod_fx**: polynomial reduction using $f_{64}(z)$
- Streams computed **y output** via AXI Stream

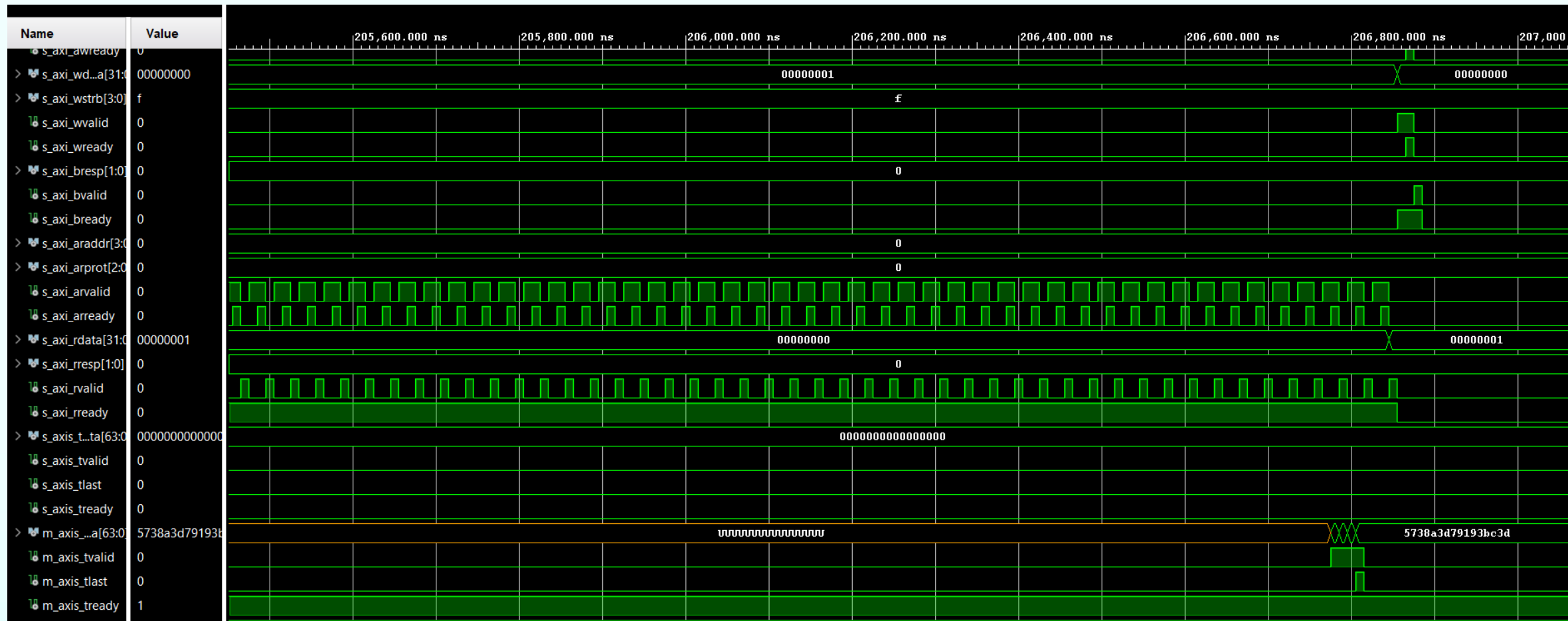


Hardware Details

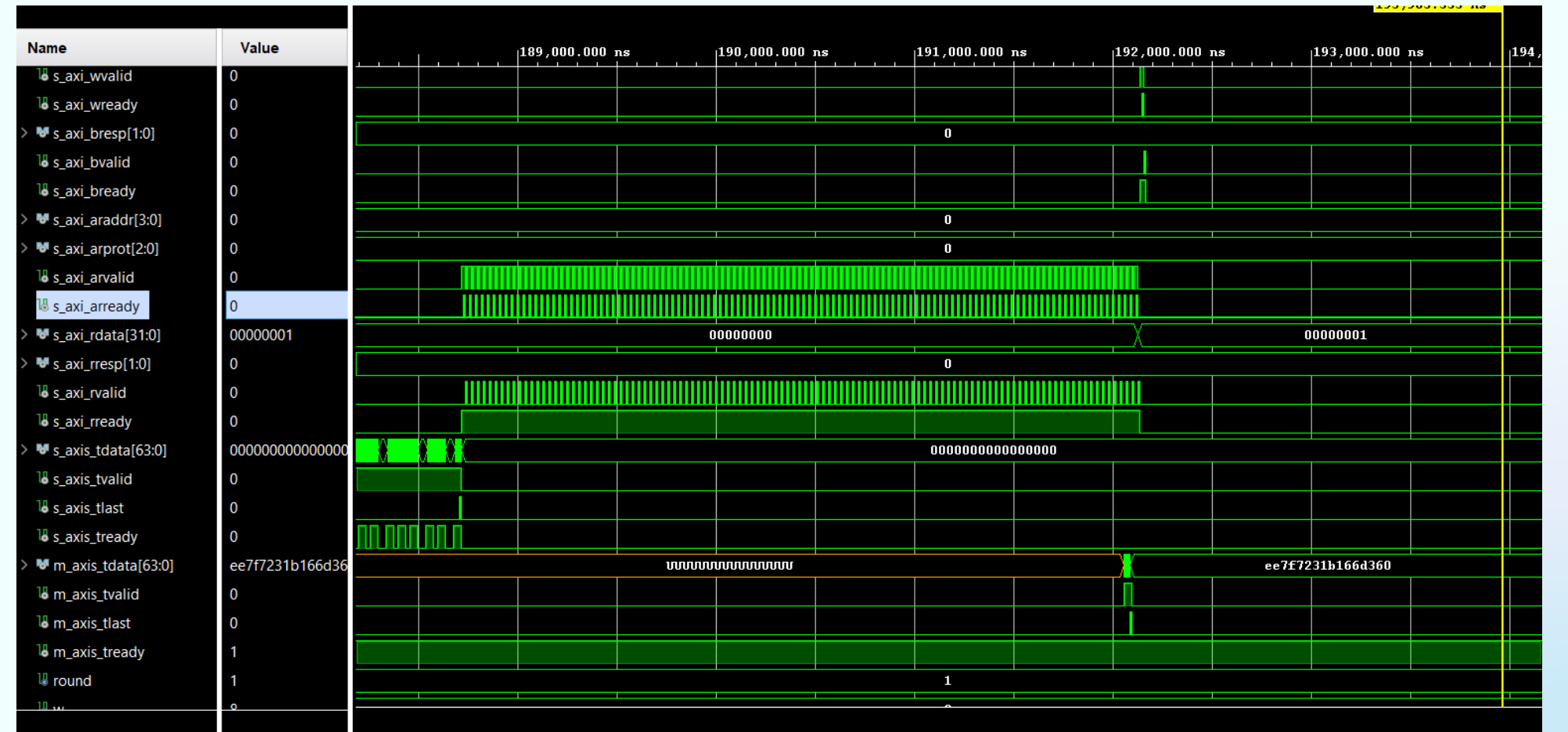
quad_accumulator



Behavioral Simulation



```
Note: calc asserted - starting combined SIG+EPK stream
Time: 145 ns Iteration: 0 Process: /mayo_verify_accelerator_tb/stimulus_proc File: C:/Users/Vakat/Downloads/code/hw_test/mayo_verify_acc
7778E2D96BB26A2E
62193C2A66253920
80D7551B4A8CF7ED
5738A3D79193BC3D LAST
Note: done observed through AXI-Lite status
Time: 206855 ns Iteration: 0 Process: /mayo_verify_accelerator_tb/stimulus_proc File: C:/Users/Vakat/Downloads/code/hw_test/mayo_verify_
Note: Simulation complete
Time: 206895 ns Iteration: 0 Process: /mayo_verify_accelerator_tb/stimulus_proc File: C:/Users/Vakat/Downloads/code/hw_test/mayo_verify_
```



```
Note: calc asserted - starting combined SIG+EPK stream
Time: 145 ns   Iteration: 0   Process: /mayo_verify_accelerator_tb/stimulus_proc   File: C:/Users/Vakat/Downloads/7B045FF3A573D0D1
690D6FE01E8E9896
F819E9CC8681645B
EE7F7231B166D360 LAST
Note: done observed through AXI-Lite status
Time: 192135 ns   Iteration: 0   Process: /mayo_verify_accelerator_tb/stimulus_proc   File: C:/Users/Vakat/Downloads/7B045FF3A573D0D1
690D6FE01E8E9896
F819E9CC8681645B
EE7F7231B166D360 LAST
Note: Simulation complete
Time: 192175 ns   Iteration: 0   Process: /mayo_verify_accelerator_tb/stimulus_proc   File: C:/Users/Vakat/Downloads/7B045FF3A573D0D1
690D6FE01E8E9896
F819E9CC8681645B
EE7F7231B166D360 LAST
run: Time (s): cpu = 00:00:02 ; elapsed = 00:00:07 . Memory (MB): peak = 5228.777 ; gain = 0.000
```

Timing Summary

Design Timing Summary					
Setup		Hold		Pulse Width	
Worst Negative Slack (WNS): 2.496 ns		Worst Hold Slack (WHS): 0.011 ns		Worst Pulse Width Slack (WPWS): 8.750 ns	
Total Negative Slack (TNS): 0.000 ns		Total Hold Slack (THS): 0.000 ns		Total Pulse Width Negative Slack (TPWS): 0.000 ns	
Number of Failing Endpoints: 0		Number of Failing Endpoints: 0		Number of Failing Endpoints: 0	
Total Number of Endpoints: 28900		Total Number of Endpoints: 28900		Total Number of Endpoints: 11200	
All user specified timing constraints are met.					

Design Timing Summary					
Setup		Hold		Pulse Width	
Worst Negative Slack (WNS): 2.873 ns		Worst Hold Slack (WHS): 0.016 ns		Worst Pulse Width Slack (WPWS): 8.750 ns	
Total Negative Slack (TNS): 0.000 ns		Total Hold Slack (THS): 0.000 ns		Total Pulse Width Negative Slack (TPWS): 0.000 ns	
Number of Failing Endpoints: 0		Number of Failing Endpoints: 0		Number of Failing Endpoints: 0	
Total Number of Endpoints: 29273		Total Number of Endpoints: 29273		Total Number of Endpoints: 11331	
All user specified timing constraints are met.					

Resource Utilization

Resource	Round 1 (R1)	Round 2 (R2)
Slice LUTs	15,169 / 17,600 (86.2%)	15,748 / 17,600 (89.5%)
Slice Registers	10,795 / 35,200 (30.7%)	10,926 / 35,200 (31.0%)
Block RAM Tiles	23 / 60 (38.3%)	23 / 60 (38.3%)

Performance Breakdown

Operation	Time
mayo_expand_pk	73.9 ms
deriveT	0.17 ms
DMA TX (SIG + EPK)	1,116.7 ms
Hardware Compute	~1.2 μ s
Total	~1,191 ms

Verification Results

Known Answer Tests (KAT) — 10 Cases per Instance

- R1 on-board UART test (Zybo Z7-10) — 10 / 10 PASS
- R2 on-board UART test (Zybo Z7-10) — 10 / 10 PASS

Conclusion

- FPGA-based hardware accelerator for MAYO post-quantum signature verification
- PS/PL co-design on Zynq Z7-10 — ARM handles crypto expansion, PL evaluates GF(16)
- GF(16) quadratic evaluation fully parallelized → $\sim 1.2 \mu\text{s}$ hardware compute
- Supports both MAYO Round 1 and Round 2, Security Level 1 (Set 2)
- DMA transfer (1,116.7 ms) is current bottleneck — dominates total latency

Thank you